

Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing

Zeja Lin
Sun Yat-sen University
Guangzhou, China
linzj39@mail2.sysu.edu.cn

Hongxin Xu
Sun Yat-sen University
Guangzhou, China
xuhx56@mail2.sysu.edu.cn

Guanyi Chen
Sun Yat-sen University
Guangzhou, China
chengy259@mail2.sysu.edu.cn

Zhiguang Chen
Sun Yat-sen University
Guangzhou, China
chenzhg29@mail.sysu.edu.cn

Yutong Lu
Sun Yat-sen University
Guangzhou, China
luyutong@mail.sysu.edu.cn

Xianwei Zhang*
Sun Yat-sen University
Guangzhou, China
zhangxw79@mail.sysu.edu.cn

Abstract

Modern large language model (LLM) serving systems confront inefficient GPU utilization due to the fundamental mismatch between compute-intensive prefill phase and memory-bound decode phase. While current practices attempt to address this by organizing these phases into hybrid batches, such solutions create an inefficient tradeoff that sacrifices either throughput or latency, leaving substantial GPU resources underutilized. For this, we identify two key root causes: 1) the prefill phase suffers from suboptimal compute utilization due to wave quantization and attention bottlenecks, and 2) hybrid batching disproportionately prioritizes latency over throughput, wasting both compute resources and memory bandwidth. To mitigate the issues, we present *Bullet*, a novel spatial-temporal orchestration system that eliminates these inefficiencies through fine-grained phase coordination. *Bullet* enables concurrent execution of prefill and decode requests, while dynamically provisioning GPU resources based on real-time performance modeling. By integrating SLO-aware scheduling and adaptive resource allocation, *Bullet* maximizes GPU utilization without compromising latency targets. Experimental evaluations on real-world workloads demonstrate that *Bullet* delivers 1.26 $\times$ average throughput gains (up to 1.55 $\times$) over state-of-the-arts, while consistently meeting latency constraints.

1 Introduction

GPUs have become the predominant computing platform for large language model (LLM) services [21, 24, 41], powering a wide range of applications with varying computational and latency demands [1, 31]. As these applications continue to grow in scale and complexity, maximizing GPU utilization has become crucial for elevating service quality [37, 60]. In response, a plethora of systems have been developed to optimize different aspects of LLM serving, such as kernel-level performance [20, 35, 44], scheduling strategies [6, 43, 59], and parallelization techniques [51].

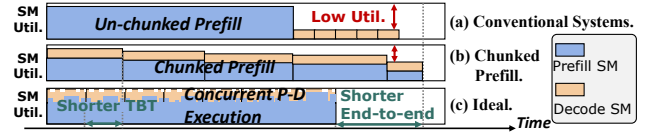


Figure 1. Request-level computation of conventional systems (top), chunked prefill (middle) and ideal (bottom).

However, the divergent computational characteristics of LLM inference make high GPU resource utilization particularly challenging [60]. In detail, the workflow consists of a computationally intensive *prefill* phase that processes all inputs in parallel, succeeded by a memory-bound *decode* phase that generates tokens sequentially. These two contrasting phases lead to a structural imbalance in the utilization of computational resources and memory bandwidth. The dynamic nature of workload patterns further exacerbates this imbalance, forcing GPUs to alternate between these complementary resource utilization states. Moreover, the distinct characteristics of the two phases inherently introduce a **throughput-latency tradeoff** [6]. For example, conventional systems [35] (Figure 1a) prioritize the prefill, which thus increases the size of subsequent decode batches. While this improves throughput, it also penalizes latency, since prefill operations monopolize GPU resources. Therefore, an ideal system necessitates fine-grained control to balance throughput and latency without elevated GPU utilization, and is non-trivial to achieve.

Prior attempts have sought to address phase imbalance either by isolating prefill and decode across different GPUs, or by coordinating them on the same device through *spatial* multiplexing. Prefill-decode disaggregation [21, 43, 59] represents the first approach, physically separating the two phases across dedicated GPU groups. However, such systems require careful tuning of GPU allocations for each phase, tailored to specific workload patterns [21], and struggle to reconfigure quickly under fluctuating request loads [59]. Furthermore, they impose heavy demands on high-bandwidth interconnects to support frequent state migration [45], significantly restricting the deployment scenarios. In practice, chunked

*Corresponding author.

prefill [6] (Figure 1b) has been pervasively adopted in production systems [35, 38, 47, 58]. This method leverages a fixed token budget to combine prefill and decode requests into *hybrid batches*, with longer sequences being partitioned into chunks to fit within capacity. While tunable chunk sizes provide a degree of latency control, the approach inevitably compromises throughput [6, 44]. Smaller chunk sizes often underutilize GPU capacity, while larger ones negate latency improvements, reflecting the inherent trade-off in balancing efficiency and responsiveness.

Although throughput-latency tradeoff in LLM serving has been extensively studied [6, 59], we identify a critical limitation that, existing approaches often underutilize GPUs, resulting in suboptimal balance between latency and hardware efficiency. First, the attention exhibits relatively low compute utilization compared to linear layers (§2.2) despite optimized implementations [20]. Second, the throughput-latency tradeoff in chunked prefill exhibits sub-linear scaling with chunk sizes that prolongs execution time for successive chunks (§2.3). Third, while spatial sharing has the potential to exploit the natural complementarity between compute-intensive prefill and memory-bound decode phases through concurrent execution (Figure 1c), effective orchestration is required. Without careful coordination, contention and latency violations remain significant risks. These observations underscore the need for dynamic coordination to fully exploit GPU performance. To address GPU under-utilization and balance the trade-off between throughput and latency, we propose *Bullet*, an LLM serving system that saturates GPU resources through **spatial-temporal orchestration with fine-grained resource provisioning**. *Bullet* proactively monitors request progress and dynamically adjusts resource provision to sustain high utilization while satisfying latency requirements. The system achieves such efficient execution with four key components: 1) a performance estimator (§3.2) with an accurate analytical model of low profile and runtime overhead; 2) an SLO-aware task scheduler (§3.3) that dynamically adjusts prefill and decode requests to balance throughput and latency; 3) a computational resource manager (§3.4) that offers lightning yet precise resource configuration; 4) a concurrent execution engine (§3.5) that enables asynchronous CPU control flow and GPU execution.

Bullet achieves optimal resource provisioning via an SM-scaling roofline model that captures contention between co-executed prefill and decode kernels with varying computational resource partitions. Calibrated with minimal samples, the model is capable of continuous refinement with online data. Layer-wise prefill scheduling monitors request progress and SLO compliance, reconfiguring resources through lightweight compute-unit masking [7]. With microsecond-level operational overhead, *Bullet* maintains both precise control and high efficiency throughout execution. *Bullet* is available at <https://github.com/zejia-lin/Bullet>.

In summary, the paper makes the following contributions:

- We identify the inefficiencies that hinder GPU utilization in existing LLM serving systems while navigating throughput-latency tradeoffs, and systematically analyze opportunities for boosting utilization.
- We establish accurate modeling for spatial-temporal shared phases, and introduce fine-grained control mechanisms for latency-aware resource provisioning.
- We design and implement an LLM serving system to effectively integrate the proposed techniques into readily available frameworks.
- Experimental evaluations on real-world workloads show that *Bullet* outperforms state-of-the-arts in both latency and throughput while achieving significantly higher GPU utilization.

2 Background and Motivation

2.1 LLM Computational Workflow

Recent LLMs [24, 41, 53] are mainly built by stacking Transformer blocks [49], with each containing four core components: QKV-projection layer, self-attention computation, Output-projection layer and multi-layer perception (MLP). These components are primarily implemented through general matrix multiplications (GEMMs), with element-wise operations interspersed between layers. Among them, the self-attention specifically operates on query, key, and value matrices produced by the QKV-projection. Its computational efficiency has been significantly enhanced through optimized kernel implementations like FlashAttention [20].

The LLM inference pipeline operates through two distinct computational phases. During the initial prefill phase, the system processes all input tokens in parallel to generate the first output token, with the latency measured as time-to-first-token (TTFT). This compute-intensive stage performs full attention computations across the entire sequence while building the KV cache to store intermediate key-value states. Subsequently, the decode phase generates tokens sequentially, with each iteration consuming only the most recent output token to produce the next. The average iteration latency is termed time-per-output-token (TPOT). Unlike prefill, this memory-bound phase primarily retrieves data from the KV cache and performs relatively lightweight computations for the current token's transformations.

2.2 GPU Utilization Principles

2.2.1 Execution Model and Theoretical Performance Bound. Modern GPUs employ a hierarchical architecture with hundreds of streaming multiprocessors (SMs), each containing general-purpose cores and specialized matrix units like Tensor Cores [17]. GPU's grid-block-thread programming model [18] aligns with this architecture, where kernels are organized into grids of thread blocks (TBs) that

each manage thousands of cooperating threads. Upon kernel launch, it enters an asynchronous task queue (termed *stream* in CUDA[18]) for scheduling. The hardware scheduler retrieves kernels from these queues and dispatches them across SMs, enabling the concurrent kernel execution (CKE) [27, 40, 50] from different streams when required resources are satisfied.

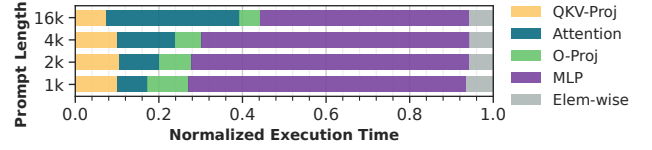
During execution, multiple TBs can reside on the same SM to share its registers, shared memory, and thread slots. The TBs per SM can be obtained via hardware vendor’s runtime APIs [2, 18]. The SM executes the *warps* (32-thread groups) in successive *wavefronts* to interleave different instructions and maximize hardware utilization. However, if the number of TBs is not evenly divisible by the number of SMs, a workload imbalance situation called *wave quantization* [15, 36] occurs. Some SMs finish early and remain idle while waiting for others to complete the tail wave. Formally, given a kernel with g TBs, N SMs, and b TBs per SM, the kernel demands $w = \lceil g/(b \cdot N) \rceil$ waves to complete. In the final wave, the TBs are distributed unevenly due to Most-Room Policy [9, 27], resulting in only $tail = \lceil g/b - N \cdot (w - 1) \rceil$ SMs active. Therefore, the corresponding ratio of idled SM cycles can be quantified as $idle = (N - tail)/(Nw)$.

GPU kernels typically use power-of-2 grid sizes to match data dimensions, but this conflicts with non-power-of-2 SM counts in GPUs. For example, 108 for Nvidia A100 [16] and 132 for H100 [17]. Therefore, wave quantization remains an open issue [34, 42] across diverse kernels. This inefficiency is particularly pronounced in Transformer’s self-attention [20, 49] and small-shaped GEMMs for short input sequences or small *chunked prefill-sizes* (§2.3.1) [6]. Additional underutilization cause arises from memory-bound kernels like LLMs’ decode phases and element-wise operators, which idle compute resources during frequent memory accesses.

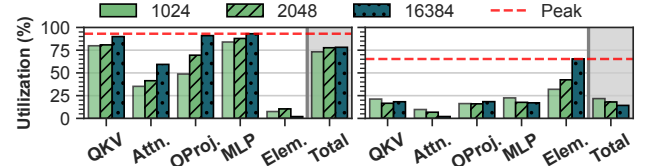
2.2.2 LLM Kernel Characteristics. We quantify LLM serving efficiency by analyzing execution time (Figure 2a), hardware utilization (Figure 2b,c), and the theoretical waste caused by wave quantization (Table 1). While MLP operations achieve up to 92% compute utilization, complete Transformer layers sustain only 70%-76% due to compounding inefficiencies. For shorter sequences, severe wave quantization in GEMMs creates substantial underutilization, as evidenced by the O-proj’s measured 49% and 70% utilization, respectively. This result closely agrees with the theoretical bound of 59% and 79% ($100 - idle\%$ in Table 1). The attention kernels,

Table 1. Theoretical SM idle ratio (%) caused by wave quantization effects, normalized to kernel/layer’s execution time.

Seq. Len.	QKV	Attn	O	MLP	Layer’s Total
1024	11.1	21.0	40.7	13.0	19.4
2048	11.1	5.2	21.0	7.6	10.4
4096	11.1	5.2	5.2	7.6	9.1
16384	1.9	0.2	0.2	0.4	0.5



(a) Execution time. (CPU overhead excluded)

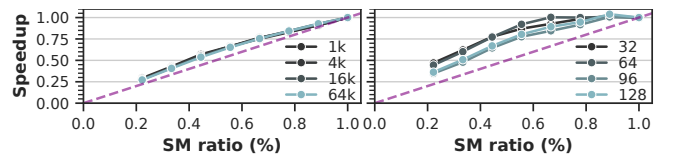


(b) Compute utilization. (c) Memory bandwidth utilization.

Figure 2. Breakdown of execution time and hardware utilization in the prefill phase of Llama-3.1-8B model on Nvidia A100 GPU. The aggregate utilization per layer remains below peak sustainable capacity (red line).

even with optimized implementations [20], exhibit lower utilization than GEMM and account for 34% of the runtime on long sequences. Together, the effects of wave quantization and attention bottleneck create persistent performance gaps between theoretical peak and achieved throughput. These gaps remain substantial regardless of sequence length, inherently constraining overall system efficiency.

2.2.3 GPU Resource Provisioning and Sharing. Naturally, compute- and memory-bound kernels are suitable to co-execute on GPUs, saturating both compute and bandwidth resources [12, 29, 46, 56]. The complementary nature of the prefill and decode phases makes them ideal for such concurrent execution. Since LLM serving systems generally necessitate adherence to service-level objectives (SLOs) of predefined latency requirements [6, 43, 59], predictable and controllable execution time over the two phases is demanded. However, current GPUs lack deterministic concurrent scheduling controls [8, 33, 40], forcing users to carefully provision resources for kernels to achieve reliable overlap [46, 56, 57]. While modern GPUs provide compute resource partitioning through Nvidia’s multi-process service (MPS) [39], precise kernel management is still required to ensure effective resource sharing while meeting SLO requirements. Figure 3 demonstrates that prefill scales near-linearly with SM count, while decode exhibits super-linear scaling. This suggests potential throughput gains from concurrent execution with properly balanced SM allocation.



(a) Prefill phase. (b) Decode phase. (CL: 2048)

Figure 3. Speedup of using partial SMs normalized to using full GPU. (Purple dashed line: linear scale.)

Takeaway 1. GPUs remain underutilized even during compute-intensive prefill. While co-locating prefill and decode saturates the resources, precise resource provisioning to orchestrate the two phases is demanded.

2.3 Biased Throughput-Latency Tradeoff

2.3.1 Chunked Prefill Workflow. As shown in Figure 4, chunked prefill [6] achieves low TPOT by leveraging a fixed token budget to concatenate the prefill and decode tokens into a *hybrid batch*, executing in a lockstep fashion (②). Given a chunk size of cs , the hybrid batch is first filled with ds active decode requests first, and allocates the remaining $cs - ds$ tokens to the prefill sequences. Sequences sl exceeding this residual capacity are split into chunks, leaving residual tokens processed in subsequent iterations. Therefore, the prefill completion requires $N = \lceil sl / (cs - ds) \rceil$ iterations. This forces $N \cdot (N + 1) / 2$ times KV cache reloads as each new chunk *must* reprocess previous chunks' cached states.

Due to the lock-step execution of the hybrid batch, a smaller chunk size effectively decreases TPOT at the cost of increased TTFT and degraded system throughput [6], while larger chunks exhibit the opposite effect. Previous works [6, 60] recognize these inherent tradeoffs and propose tuning chunk size based on workload's prefill-to-decode time ratio through manual tuning or automatic searching [5, 11].

2.3.2 Sub-optimal Hardware Utilization. Despite the throughput-latency tradeoff has been extensively studied, we highlight that such a tradeoff is biased, and the resulting performance degradation remains overlooked. **First**, as discussed in §2.2.2, chunked prefill typically uses suboptimal chunk sizes below GPU-saturating levels to prioritize low latency. This produces severe wave quantization effects [42], creating GPU bubbles (Figure 4-①). **Second**, redundant KV cache reloads required for long sequences significantly prolong attention computation time (③), further reducing GPU utilization. **Third**, these factors collectively inflate TTFT, triggering a cascading congestion effect in which queued requests stall while awaiting prefill completion, degrading overall system throughput.

Figure 5 systematically quantify the performance degradation of chunked prefill of a 16k-token sequence prefill even *without* hybrid batching decode requests. For 1k chunk

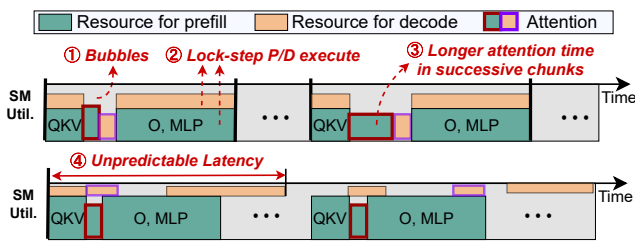


Figure 4. Kernel-level workflow of existing systems featuring chunked (top) and uncoordinated sharing (bottom). Both suffer from biased throughput-latency tradeoff.

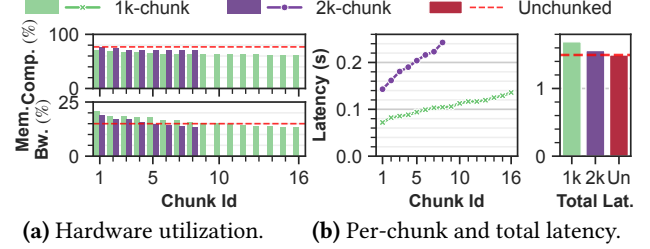


Figure 5. Per-chunk GPU utilization and latency across varying chunk sizes, all exhibiting degraded throughput.

size, a progressive 10% drop in compute efficiency (from 71% to 61%) across successive chunks is witnessed in Figure 5a, which falls substantially below the 77% achievable peak. This under-utilization stems from redundant KV cache reloading in chunked attention, which also causes the final chunk's processing time to 1.9× than that of the initial chunk. Consequently, per-chunk latency scales linearly by chunk counts and increases total prefill latency by 1.13× compared to unchunked execution. While a larger chunk size of 2k partially mitigates utilization drops from -18% to -7%, the average per-chunk latency increases by 1.86×, significantly diminishing the TPOT improvements that motivated chunked prefill. This fundamental tension between maintaining high hardware utilization and minimizing TPOT creates an intractable optimization in chunked prefill.

2.3.3 Existing Optimizations. Despite recent optimizations of chunked prefill, the technique remains intrinsically limited. PodAttention [44] fuses prefill and decode attention kernels to accelerate hybrid-batch attention, yet this operator-specific optimization does not mitigate the fundamental latency penalties imposed by chunking (§2.3.2). Nanoflow [60] designs a fixed-size pipeline that uses CUDA streams to overlap chunked attention and linear layers with careful inter-kernel synchronizations. However, the latency of chunked attention grows by successive chunks, which eventually eliminates the chances of overlapping. Apart from chunked prefill-based approaches, MuxServe [23] (Figure 4) decouples prefill and decode phases into separate processes, leveraging MPS [39] with manually configured, fixed SM quotas to enable spatial sharing. This coarse-grained static

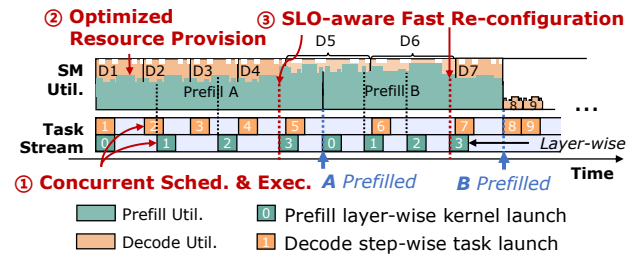


Figure 6. Dynamic spatial-temporal orchestration of concurrently executed prefill and decode tasks.

partitioning leads to unpredictable latency despite saturating the GPU. Therefore, neither solution fully resolves the throughput-latency tradeoff under varying serving demands.

Takeaway-2. *Both chunked prefill and uncoordinated spatial sharing exhibit skewed throughput-latency tradeoffs. This urges for a fine-grained yet predictable execution mechanism for optimal LLM serving performance.*

2.4 Opportunities and Challenges

To address the inefficiencies discussed above, an inter-phase orchestration system is demanded to achieve optimal throughput-latency balance while maximizing GPU utilization, as illustrated in Figure 6.

Concurrent Schedule and Execution. Co-executing prefill and decode eliminates redundant KV-cache reloads and fully utilizes GPU resources, but sustaining latency targets demands real-time request monitoring and kernel scheduling that respond to execution progress and system state. A layer-level, SLO-aware scheduler must tightly control execution advance to avoid inter-phase resource competition. While LLM online serving further requires low-overhead control plane and support for rapid reconfiguration, rendering these challenges non-trivial.

Optimized Resource Provision. By carefully partitioning available SM resources, memory-bound decode kernels may efficiently co-execute with compute-intensive prefill kernels. However, LLM’s multi-dimensional nature of input parameters and dynamic inter-kernel contention creates a complex optimization space. This necessitates precise latency modeling across varying SM provisions and efficient exploration algorithms to identify optimal configurations.

SLO-aware Rapid Re-configuration. In response to runtime workload dynamics, resource allocation between prefill and decode phases must be re-configured. These frequent adjustments demand near-zero-overhead reconfiguration to maintain efficiency. Although MPS [39] offers context APIs [18] to adjust different SM quotas, it incurs significant memory overhead when applied to LLM serving scenarios due to dynamic inputs. An even more lightweight approach is demanded to maintain for instantaneous switching between optimized resource partitions.

3 Design

3.1 Overview

Figure 7 illustrates Bullet’s workflow for concurrent prefill-decode execution with dynamic and fine-grained resource provisioning, with all components operating at microsecond-level overhead. Bullet comprises four key components around the scheduling, resource partitioning and execution: performance estimator, SLO-aware task scheduler, resource manager, and concurrent execution engine. The *performance estimator* (§3.2) ❶ first builds an analytical model for the served LLM, augmented with lightweight offline samples. The model

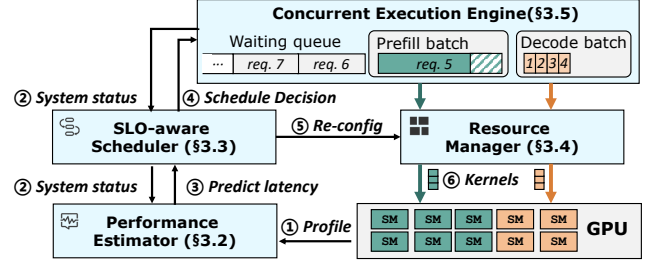


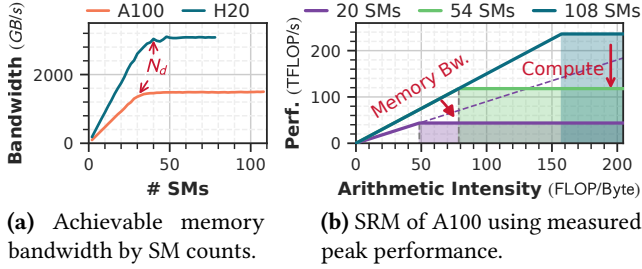
Figure 7. Workflow of Bullet. (Numbers: dataflow order.)

provides precise latency predictions across different configurations of co-executing prefill and decode batch sizes under varying resource allocations. At runtime, the *SLO-aware task scheduler* (§3.3) acts as the central coordinator to enable GPU sharing between prefill and decode tasks via spatial-temporal scheduling. During every layer-wise scheduling cycle, the scheduler ❷ proactively retrieves system status from the *concurrent execution engine* (§3.5), monitoring request progress and ❸ evaluating potential SLO violations by *performance estimator*. Observed statistics are also used to refine the performance estimator online. The scheduler rapidly searches for ❹ an optimal resource configuration and scheduling decision that maximizes throughput while ensuring SLO compliance. *Computational resource manager* is ❺ triggered for lightning resource reconfiguration when necessary. Finally, the prefill and decode kernels are ❻ launched concurrently on the provisioned SMs. Bullet dynamically balancing the competing demands of TTFT and TPOT maintains high utilization with negligible overhead.

3.2 Performance Estimator

3.2.1 Problem Formulation. For a given LLM and hardware, the latency of co-executed prefill and decode is determined by six factors, termed as Execution State (*ES*): prefill sequence length (sl_i), batch size (pbs) and number of allocated SMs (pm), alongside decode context length (cl_i), batch size (dbs) and SMs (dm). Enumerating the millions of *ES* combinations for profiling is infeasible. Therefore, we build a lightweight analytical model with a minimal profile and runtime overhead. First, we propose SM-scaling roofline model (SRM) to derive single kernel latency under partitioned SMs without interference. Second, memory-subsystem contention is quantified when concurrent kernels are isolated in distinct SMs. Finally, the model is augmented with minimal sampled data for calibration. At runtime, the model executes within microseconds and continuously refines with online statistics.

3.2.2 SM-scaling Roofline Model (SRM). We examine the performance of compute, memory access, and network communication when only N_p of the SMs are available. Theoretical compute performance scales linearly as $C_p = C_{peak} \cdot N_p / N$. Memory and network bandwidth exhibit proportional



(a) Achievable memory bandwidth by SM counts. (b) SRM of A100 using measured peak performance.

Figure 8. Modeling peak performance by number of SMs.

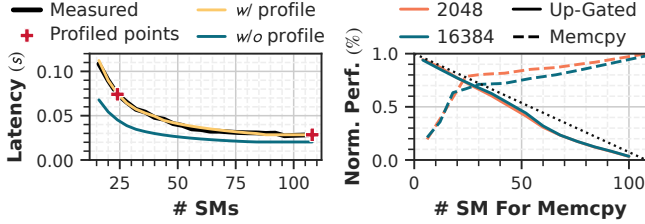


Figure 9. Calibrating estimated decode latency with only 2 samples.

Figure 10. Normalized performance of co-executed UG and memory copy.

scaling until reaching inflection points N_d and N_w , where the SMs generate sufficient traffic to saturate the respective peak bandwidth D_{peak} and W_{peak} . Figure 8a illustrates the throughput of a memory-copy kernel, showing the inflection points of Nvidia A100 and H20 GPUs.

Given a kernel with $flop_k$ operations and mem_k bytes of memory transactions, we construct an SM-scaling roofline model (SRM) in Equation 1 to estimate the theoretical latency on N_p SMs. In the example of Figure 8b, the memory bandwidth inflection point $N_d = 30$. When using 54 SMs, the attainable memory bandwidth remains at its peak, maintaining the original roofline slope while lowering the plateau. For 20 SMs, both the slope and plateau decline.

$$\begin{cases} T'_{k,p} = flop_k \cdot \min(flop_k/mem_k \cdot D_p, C_p)^{-1} \\ C_p = C_{peak} \cdot N_p/N; D_p = D_{peak} \cdot \min(1, N_p/N_d) \end{cases} \quad (1)$$

For each Execution State ES , we compute the arithmetic intensity of every LLM kernel with llm-viewer [54], apply SRM to obtain its baseline latency $T'_{k,p,ES}$ on N_p SMs, and aggregate these values to yield the total LLM latency $T'_{p,ES}$. Since practical execution rarely matches the roofline bound, Equation 2 derives a scaling factor for calibrating SRM:

$$\alpha_{p,ES} = T_{p,ES}^{measured} / T'_{p,ES} \quad (2)$$

The factor is then extrapolated to unmeasured configurations, since the utilization pattern is near-linear between similar kernel inputs [10, 12, 26]. As shown in Figure 9, only two samples are sufficient to model the decode latencies by varying SMs. This demonstrates effective estimation aligned with kernel characteristics without extensive profiling.

3.2.3 Modeling Contention. When kernels execute on isolated SMs to prevent compute contention, memory subsystem and network contention persist. We verify that kernel latency remains stable under such interference, even if underlying hardware scheduling and inter-kernel resource competition vary. Extensive evaluation of 7360 concurrent ES on A100 (serve Llama3.1-8B [24]) and 8×H20 (serve Qwen3-32B [53]), each repeated for 30 runs, observes that 95% of latency measurements deviate less than $\pm 6.8\%$ from the mean of the respective repeated runs. This tight distribution confirms the stability of kernel latency under varying prefill-decode interference patterns, which can be used as a reliable metric for contention modeling.

The worst-case memory subsystem interference can be quantified by co-executing a memory copy kernel on N_p SMs and an up-gated layer (UG), which is a large-shaped GEMM in LLM, on the residual $N - N_p$ SMs. This is because prefill and decode kernels typically exhibit lower resource utilization than both UG and memory copy. Figure 10 shows that UG exhibits marginal performance degradation ($< 8\%$) when using more than 60% SMs. Therefore, the latency of prefill kernels can be reliably calibrated via Equation 2 with minimal sample size. Conversely, memory-copy throughput decreases as UG sequence lengths grow. To model decode latencies, we measure the attainable bandwidth $D_{p,sl}$ when concurrently executed with sl -length prefill to update SRM. Equation 2 is then applied for further refinement, since the memory interference pattern scales linearly with kernel utilization characteristics [12, 52].

Inter-phase network contention is low since the traffic scales with either sl or dbs , where dbs is relatively small. The memory subsystem impact of network transfers is inherently lower than memory copy, and prefill-to-decode interference is already accounted for in the end-to-end $D_{p,sl}$ calibration.

3.2.4 Profiling and Online Calibration. During offline profiling, Bullet records compute, memory, and network performance across varying SM counts in a single sweep to construct SRM, then measures prefill and decode latencies for a sparse set of ES values as calibration. Continuous online data collection for recalibration is straightforward. Leveraging only Equation 2 and linear extrapolation, the runtime overhead for update and prediction is negligible.

Table 2 shows that calibrated SRM maintains high accuracy even for inputs beyond the sampled range while keeping profiling overhead low. This precision is sufficient for reliable real-time scheduling optimization.

3.3 SLO-aware Task Scheduler

3.3.1 Scheduling Workflow. Each of the prefill and decode *concurrent execution engine* (§3.5) runs a scheduler autonomously. At every step, the scheduler reads system status from the global metadata buffer (§3.5.1), forecasts latencies with *performance estimator*, and reorders pending requests.

Table 2. Mean absolute relative error of SRM on 7360 data.

Phase (Data Range)	#Samples (Range)	MAPE
Prefill	18 ($sl \leq 16k$)	9.1%
($sl \leq 32k$)	30 ($sl \leq 32k$)	5.3%
Decode	308 ($cl \leq 16k, bs \leq 128$)	10.5%
($cl \leq 32k, bs \leq 256$)	364 ($cl \leq 32k, bs \leq 256$)	7.7%

Upon detecting potential SLO violations, the scheduler greedily searches for the optimal configuration and invokes *computational resource manager* (§3.4) to repartition SMs.

As illustrated in Figure 11, for prefill scheduler, a fixed number of layers is launched per step, and synchronizes for CPU (teal triangle) to make subsequent scheduling decisions. This enables fine-grained control over prefill progress temporarily for rapid adaptation to system fluctuations. Conversely, decode scheduler issue kernels as a single CUDA Graph [18] to eliminate the launch overhead of small kernels.

The primary scheduling objective is to prioritize prefill while respecting decode SLOs, since shorter prefill latency enlarges decode batch size and raises system throughput [6]. During concurrent operation, the decode phase is provisioned with the minimum SM counts that satisfy SLO. As the final prefill layer approaches completion, additional SMs are allocated to decode phase to facilitate a smooth transition between co-running prefill/decode and decode-only. During reconfiguration, Bullet eliminates inter-phase synchronization by partially sharing SMs between phases rather than idling unused resources. Although a perfect non-overlapping allocation is challenging, layer-wise prefill scheduling confines such compute resource interference to minimal, predictable regions.

3.3.2 Request Scheduling and Resource Provisioning.

To facilitate SLO-aware scheduling, Bullet monitors the system state defined as $S = (ES, PS, RS)$, where ES denotes the execution state (§3.2.1), PS the prefill progress, and RS the per-request latency metrics. Prefill state PS comprises the queuing-request set Q , the in-flight request set P , and the executed layer count L_{exe} . For every request i , arrival time a_i and decode-start time d_i are recorded. At time t , the prefill scheduler estimates GPU execution times for all requests in $P \cup Q$ under current ES , while the decode scheduler predicts next-step latency and updates the corresponding TPOT. These estimations are then written to the global metadata buffer, allowing each scheduler perceive the holistic state.

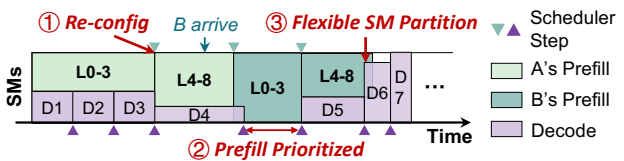


Figure 11. Asynchronous scheduler launches layer-wise prefill kernels and step-wise decode graphs, monitoring latencies and dynamically reallocating SMs.

Algorithm 1 outlines the prefill scheduler, and the decode scheduler follows analogously. The algorithm continuously monitors execution progress and updates latency estimates (lines 2-4). Requests in the waiting queue are reordered by ascending predicted latency (line 5) when the reordering does not violate TTFT SLOs for pending requests. This reduces average TTFT without starvation. To start a new prefill step (line 10-13), requests are batched until reaching arithmetic intensity limits under the current execution state. When provisioning resources, prefill phase is prioritized and provisioned with more SMs unless TPOT is compromised (line 15). In extreme cases of high request load, the decode phase would be temporarily suspended (Figure 11-②) if TPOT SLO is still met. When both SLOs cannot simultaneously be satisfied, indicating the system is beyond maximum capacity, a balanced SM ratio is enforced to limit excessive latency in either phase (line 14).

3.4 Computational Resource Manager

While MPS [39] with CUDA Green Context [3] supports SM partitioning, its memory overhead exceeds 700MB for only 4 static policies in LLM serving [19], rendering it impractical for Bullet's fine-grained control. For flexible low-overhead SM provisioning, we employ a library of SM masking techniques [7, 8]. The `libsmctrl_set_stream_mask` API sets the metadata of a CUDA stream [18] to restrict all kernels

Algorithm 1: SLO-aware Scheduling for Prefill

Input: System state: S , TTFT/TPOT SLO: Γ_p/Γ_d

Output: Next requests & layers to run: $next_tasks$

```

1 Function Schedule( $S$ ):
2    $ttft \leftarrow \text{ESTIMATELATENCY}(S)$ 
3    $\text{WRITEGLOBALBUFFER}(ttft)$ 
4    $tpot \leftarrow \text{READGLOBALBUFFER}()$ 
5    $\text{SORTBYLEASTESTIMLATENCY}(Q)$ 
6   if  $P \neq \emptyset$  then
7      $satisfy \leftarrow \text{req}.ttft \leq \Gamma_p, \text{req} \in P$ 
8      $next\_tasks \leftarrow P$ 
9   else
10     $satisfy \leftarrow P90(ttft) \leq \Gamma_p$ 
11     $next\_tasks \leftarrow \emptyset$ 
12    while  $\text{ARITHINTEN}(next\_tasks, S.ES) < peak$  do
13       $next\_tasks.append(Q.pop())$ 
14    if not  $satisfy$  and  $P90(tpot) > \Gamma_d$  then
15       $\text{SETBALANCEDSM}(S, ttft, tpot)$ 
16    else if  $P90(tpot) \leq \Gamma_d$  then
17       $\text{REDUCEDECODESM}(S, ttft, tpot)$ 
18    else if  $P90(tpot) > \Gamma_d$  then
19       $\text{REDUCEPREFILLSM}(S, ttft, tpot)$ 
20    return  $next\_tasks, L_{exe} + L_{step}$ 

```

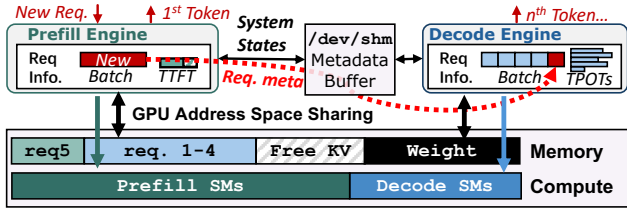



Figure 12. Concurrent execution engine with shared KV cache/weight GPU address spaces, exchanging system states and request metadata via OS-managed shared memory.

launched within it to execute on a specified SM subset. Section 4.3.3 validates that this approach adds only microsecond-level runtime overhead and zero extra memory. Similar interfaces exist on other platforms, exemplified by AMD’s `hipExtStreamCreateWithCUMask` [2].

Bullet creates a CUDA stream in each *concurrent execution engine* for prefill and decode. Whenever the scheduler issues a repartitioning command, the system immediately invokes the `libsmctrl` API to reconfigure the corresponding stream, thereby restricting all subsequent kernels to the newly provisioned SMs. This instantaneous configuration (Figure 11-①③), supports rapid adaptation to dynamic system states with flexible SM allocation.

3.5 Concurrent Execution Engine

3.5.1 Engine Architecture. Figure 12 illustrates the concurrent execution engines for prefill and decode, each residing in a separate process and driven by the corresponding scheduler. Both engines share a CPU buffer and unified GPU memory pool. The CPU buffer is implemented as OS-managed shared memory, stores global system states and employs compact control bits to indicate data availability, enabling low-latency status exchange. For GPU memory management, Bullet employs a dedicated initialization process that allocates model weights and KV cache [35] prior to engine launch. The resulting memory region is shared between engines via `cudaIpcGet/OpenMemHandle` API [18] with no adverse effects as documented. Equivalent facilities, such as AMD’s `hipIpcOpenMemHandle` [2], allow the same design to be deployed on other hardware. Since the address space and metadata are shared, Bullet is fully compatible with existing KV cache and prefix cache optimizations. An atomic lock serializes allocation and deallocation transactions that may be issued concurrently by both engines, ensuring correctness with minimal performance impact.

3.5.2 Execution Workflow. The prefill engine’s execution begins with receiving requests and follows the scheduling workflow in §3.3, retrieving decode-side states from the buffer, such as batch size and TPOTs. Once prefill completes, the request migrates to decode without KV-cache transfer, and only metadata is asynchronously sent to the

decode engine via ZeroMQ [55], enabling microsecond overhead. The decode engine receives newly prefilled requests and merges them into the current running batch. Output tokens are directly forwarded to the frontend server, eliminating any CPU involvement by the prefill engine. Bullet’s control plane works independently while proactively communicating through the buffer, unblocking both CPU and GPU execution. This decentralized architecture allows concurrent kernel submissions while eliminating the need for frequent synchronization compared with a centralized architecture. We implement Bullet on top of SGLang [58] v0.4.6 and PyTorch 2.6.0 with 4100 lines of Python code, and integrate a modified `libsmctrl` [7] library to optimize GPU resource allocation within the serving engine. The prefill and decode engines are implemented as SGLang’s workers, with MPS [39] enabled for spatial sharing.

4 Experimental Evaluation

4.1 Methodology

Evaluated Schemes. We compare Bullet against several state-of-the-art systems of chunked prefill-based optimization: vLLM [35] v0.8.5 (1024-chunk), SGLang [58] v0.4.6 (1024/2048-chunk), and Nanoflow [60] (1024-chunk).

Workloads Three representative datasets are used for evaluation, whose request arrivals follow a Poisson process, as widely adopted in prior works [6, 35, 60]. Figure 13 details the cumulative distribution function (CDF) of the datasets’ sequence length. The ShareGPT [48] contains real-world conversational data, Azure-Code [43] is production code completion trace released by Azure, and arXiv-Summary [14] is long-context summarization.

Models and Platforms. Experiments are conducted on three servers in Table 3 with CUDA 12.4. We evaluate dense model Llama3.1-8B/70B [24] and MoE model Qwen3-235B-A22B-FP8 [53]. All multi-GPU setups use tensor parallelism.

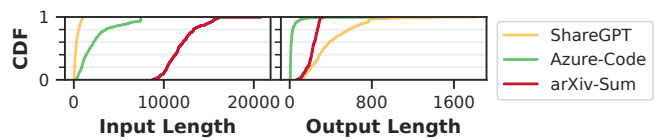


Figure 13. Workload input/output length CDF.

Table 3. Server configurations for experiments.

GPUs	Network Bw.	#SMs/GPU
8×A100-80GB	20 GB/s	108
8×H100	600 GB/s	132
8×H20	400 GB/s	78

Table 4. Workload latency requirements.

	ShareGPT	Azure-Code	arXiv-Sum
norm. TTFT	3.0 ms	1.5 ms	1.5 ms
TPOT	150 ms	200 ms	175 ms

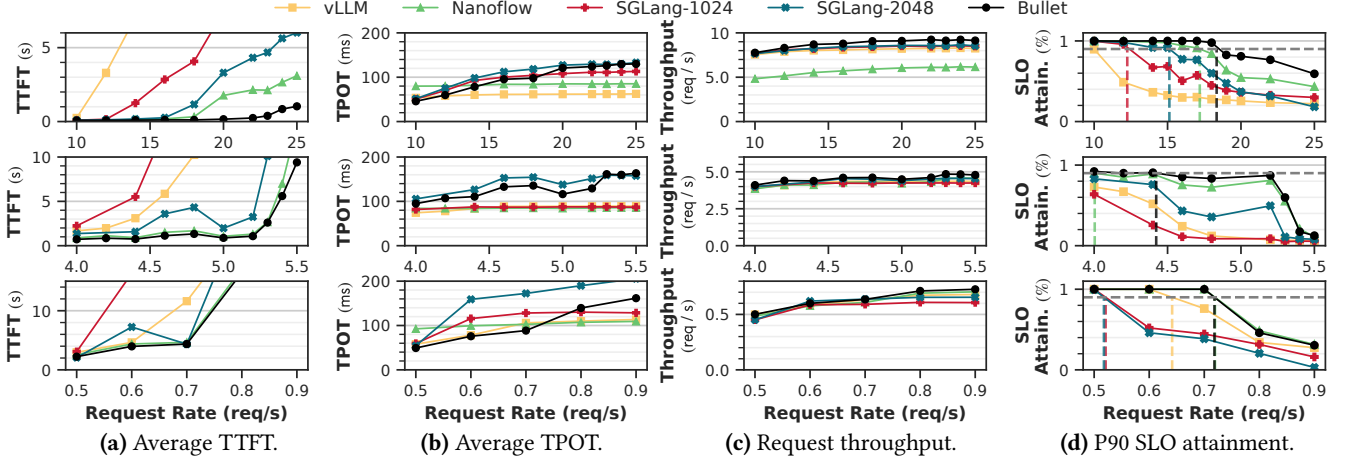


Figure 14. Performance comparison of ShareGPT (top row), Azure-Code (middle) and arXiv-Summary (bottom) datasets, evaluating average latency, throughput and SLO attainment rate of different LLM serving systems. Bullet achieves the highest throughput and SLO compliance in all workloads, attributed to remarkably decreased TTFT with modest TPOT increments.

Metrics. We report TTFT, TPOT and throughput. For SLO compliance measurement, we define as prefill and decode latency satisfying both the constraints listed in Table 4, aligned with previous works [51, 59, 60]. Due to the variation in sequence length, we use *normalized input/generation latency* established in previous works [51, 60] for SLO measurement. We collect hardware metrics using Nsight Systems [4].

4.2 End-to-End Performance

4.2.1 Single GPU Performance. Figure 14 evaluate Bullet against four baselines across three real-world workloads. Bullet demonstrates consistent throughput improvements, achieving 1.09 $\times$ average and up to 1.20 $\times$ higher throughput compared to SGLang-1024. Overall, Bullet maintains superior prefill latency enhancement (13.5 $\times$) with acceptable TPOT (0.94 $\times$) in average through dynamic SM allocation, translating to 1.86 $\times$ end-to-end speedup compared with SGLang-1024. In contrast, all the chunked prefill-based systems exhibit unacceptable TTFT degradation even in a low request rate. For example, on the ShareGPT workload, with request rate of 20req/s, the prefill latency ranges from 1.8s (Nanoflow) to 14.9s (vLLM). The key reason is that chunk prefill limits the system’s capability to process the prefill phase (§2.3.2) and creates a cascading congestion for pending requests.

4.2.2 SLO Attainment. Figure 14d presents P90 SLO attainment rate. By eliminating these bottlenecks through intelligent SM allocation and adaptive scheduling, Bullet achieves mean TTFT of 0.16s and P90 tail latency of 0.31s, which is 54.9 $\times$ and 78.5 $\times$ better to SGLang-1024. The lower prefill latency remarkably translates to larger decode batches that boost overall throughput and SLO compliance (1.49 $\times$). Bullet effectively balances the throughput-latency tradeoff

that is skewed in chunked prefill systems. While SGLang-2048 demonstrates a 3.20 $\times$ TTFT improvement and 1.18 $\times$ higher throughput than SGLang-1024, the 2048 chunk suffers from 0.78 $\times$ worse TPOT on average, confirming the imbalanced tradeoff inherent in chunked prefill. Bullet breaks this paradigm by simultaneously achieving both lower TTFT (4.2 $\times$) and better TPOT (1.20 $\times$) than SGLang-2048 through exploiting GPU utilization via concurrent prefill-decode execution. While Nanoflow achieves 2.37 $\times$ longer TTFT and 0.86 $\times$ shorter TPOT than Bullet via static kernel overlapping pipeline, it still suffers from the inherent limitations of chunked prefill that inflate TTFT tail latency. This results in 5.2% lower SLO compliance and 20.0% lower throughput compared to Bullet.

4.2.3 Multi-GPU Performance. Figure 15 depicts the generation latency of Bullet when Llama3.1-70B is deployed across eight A100 GPUs, with vLLM and SGLang using a chunk size of 2048. On the ShareGPT workload at 3.0 req/s, Bullet achieves a mean latency of 173 ms/token (223 ms/token for P99), whereas vLLM and SGLang attain 207 ms/token and 319 ms/token, respectively. For the Azure-Code dataset, where input lengths are significantly longer, Bullet’s advantages is more pronounced, sustaining 203 ms/token at

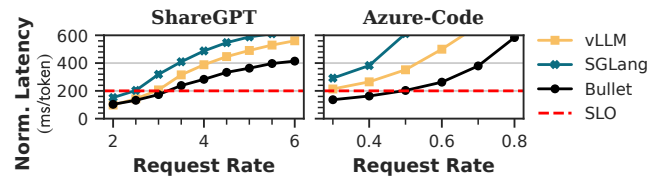


Figure 15. Mean normalized generation latency of Llama3.1-70B served on 8xA100. Bullet sustains higher request rates while remaining within the SLO constraint.

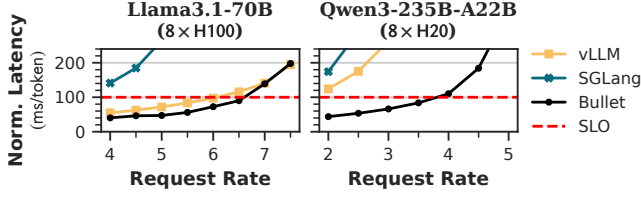


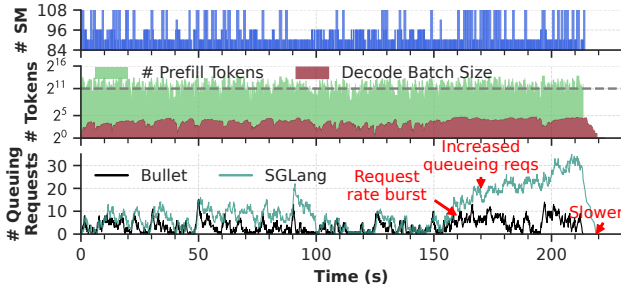
Figure 16. Mean normalized generation latency of various models across different hardware serving Azure-Code.

0.5 req/s. In contrast, vLLM and SGLang already exhibit 213 ms/token and 291 ms/token at a lower rate of 0.3 req/s.

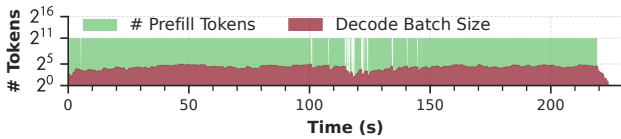
On H100 GPUs, the elevated memory bandwidth mitigates chunked-prefill overhead, yet Bullet continues outperforms SGLang and vLLM. Bullet’s concurrent handling of prefill and decode tasks leverages potential overlaps between computation, communication and memory operations with fine-grained SM control between phases. We also demonstrate Bullet’s generalization to other LLMs. For the highly sparse, FP8-quantized MoE Qwen3-235B-A22B, chunked prefill’s inherent lock-step batching impedes both prefill and decode speeds. However, Bullet orchestrates the two phases effectively. At 4.0 req/s, Bullet delivers 110 ms/token, with 1.4s TTFT and 45ms TPOT—respectively, which is 17.7 $\times$, 16.2 $\times$ and 4.2 $\times$ lower than vLLM.

4.3 Performance Breakdown

4.3.1 Dynamic SM Partition. We break down the effectiveness of dynamic resource provisioning in Figure 17 by timeline with the Azure-Code workload (5.0 req/s, Llama3.1-8B). The top row in Figure 17a demonstrates the number of SMs provisioned for the prefill phase, with each bar showing



(a) SM allocation for prefill changes dynamically according to system load (top). Concurrently processing tokens/batch size in Bullet (middle). Number of requests waiting for prefill (bottom). Bullet adaptively partition SMs to avoids request congestion on burst.



(b) SGLang’s hybrid batch status. Decode requests occupy chunk budget, degrade prefill efficiency, incurring severe queueing time.

Figure 17. System serving status of the Azure-Code workload in timeline view (request rate: 5.0 req/s).

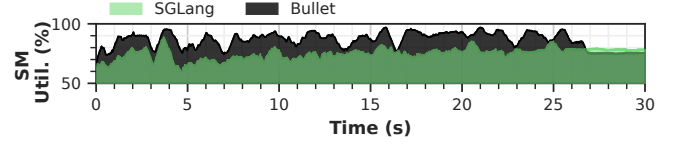


Figure 18. SM utilization of a timeslice in ShareGPT workload (20 req/s, Llama3.1-8B).

the SM count and duration. On request rate bursts (spikes in Figure 17-bottom), Bullet adaptively sets the number of prefill SMs to use full GPU, and may temporarily delay decode requests. This enables the rapid response for queuing requests, avoiding excessive waiting time. After processing the pending requests, Bullet quickly reconfigures the resources to a balance point for both phases. Instead, chunked prefill-based systems are not able to optimize such a scenario, since redundant KV cache access inflates the prefill speed. Moreover, decode batch shares the token budget with prefill tokens (Figure 17b), enforcing more iterations to complete a prefill, which further degrades performance. These factors jointly result in 4.17 $\times$ longer queuing delay for SGLang-2048. Bullet exempts from the token budget (Figure 17-middle) and orchestrates the two phases with fine-grained resource control to saturate GPU, significantly decreasing both TTFT and TPOT by 9.15 $\times$ and 1.33 $\times$, respectively.

4.3.2 GPU Utilization. We measure hardware utilization with Nsight Systems and present it in Figure 18. Between 0 s and 27 s, when the system concurrently handles prefill and decode requests, Bullet sustains an average of 86.2% active SM cycles, which is 11.2% higher than SGLang. Correspondingly, utilization of Tensor Cores rises by 11.8%, while memory-bandwidth utilization increases by 19.3%. These utilization gains directly reduce the mean end-to-end request latency from 25.8 s to 21.4 s.

4.3.3 Overheads. Table 5 quantifies the CPU overhead of Bullet’s key components. Sending and receiving meta-data only necessitates serialization and deserialization of Python objects, therefore achieving mean latency of 0.21ms. The performance prediction module incurs merely 10.2 μ s overhead by simply invoking the analytical model. For GPU resource management, SM masking enables instantaneous reconfiguration with negligible runtime overhead. These designs jointly ensure Bullet’s control plane adds minimal latency while enabling fine-grained resource optimization.

4.4 Sensitivity Studies

To study the effect of dynamic resource provisioning, we run the workloads under fixed SM configurations for prefill and allow decode to use all SMs. The results are reported in Figure 19. The SM-108 configuration (no partitioning) demonstrates severe imbalance in the Azure-Code workload. While achieving low TPOT, SM-108 suffers 1.20 $\times$ higher TTFT in average and 1.19 $\times$ worse P90 tail latency compared

Table 5. Overheads in Bullet.

	Mean	Std.	P90	P99
Metadata Send/Recv (ms)	0.21	0.44	0.89	1.54
Performance Predict (μs)	10.2	5.1	24.5	25.8
Resource Re-config (μs)	4.1	0.79	4.2	5.9

to Bullet, ultimately reducing throughput and SLO attainment by 13%. Smaller static partitions, like 84 SMs, prove even more problematic, exacerbating latency imbalance with 1.78 $\times$ worse TTFT even than chunked prefill baselines and reducing throughput by 5.9%. Therefore, there is no optimal fixed SM allocation, as smaller partitions improve TPOT but degrade TTFT and introduce tail latency violations, and vice versa. These results confirm Bullet’s dynamic approach, which adaptively orchestrates resources to simultaneously maintain balanced latency targets and maximize throughput.

4.5 Ablation Studies

Figure 20 analyzes the contributions of different components in Bullet by isolating part of the design. Naive: concurrent execution without resource provisioning or scheduling. w/Partition: adds resource provision only. w/Scheduler: Incorporates only request reordering and delayed decode. The Naive design exhibits the expected latency imbalance (§4.4), high TPOT and low TTFT attributed to unpartitioned resource contention. The w/Partition variant improves TPOT for Azure-Code but suffers unacceptable TTFT degradation from its inability to reorder pending requests. Conversely, w/Scheduler maintains comparable latency while reducing Azure-Code TPOT through contention alleviation. Only the complete Bullet design, combining both partitioning and scheduling, achieves balanced latency across all workloads.

4.6 Discussions

While Bullet shows promising improvements, we identify several operational boundaries. First, on low-compute GPUs running dense models, the limited SM budget obliges the decode phase to claim a larger ratio of SMs to saturate memory bandwidth (Figure 8a), slightly tempering concurrent-execution benefits. However, this caveat is immaterial for

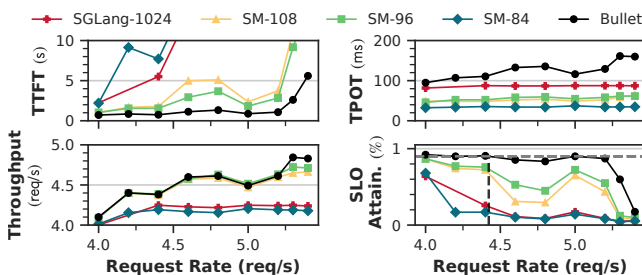


Figure 19. Sensitivity study on fixed SM counts for prefill. Static setups cause latency-throughput imbalance: more SMs cut TTFT but harm TPOT and goodput. Bullet’s dynamic SM tuning optimizes both.

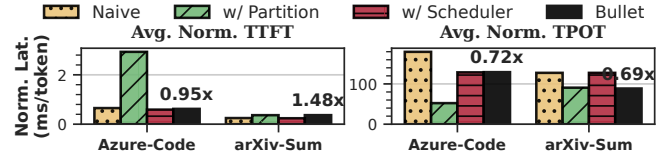


Figure 20. Ablation study of Bullet.

MoE models, whose lighter compute footprint preserves the full advantage (Figure 16). Second, Bullet’s SM isolation policy could be relaxed to partial sharing, yet such exploration is orthogonal to the paper’s core contribution of fine-grained spatial-temporal orchestration and can be retrofitted without architectural changes. Third, for specialized architectures like DeepSeek-MLA [21] where disaggregation proves fundamentally advantageous, Bullet’s co-located approach may not match dedicated solutions’ peak performance. Extending the latency model to cover new attention variants or LoRA adapters [32] is straightforward and left to future work. Bullet’s core innovations maintain broad applicability across standard LLM architectures and typical workload distributions while extensible to emerging models.

5 Related Works

LLM Serving Systems. System-level optimizations for LLM serving have been extensively explored, which can be categorized as prefill-decode co-location and disaggregation. Co-located techniques represented by chunked prefill [6, 60] improve batching efficiency but maintain lockstep prefill and decode phase execution that underutilizes hardware. Disaggregated systems [25, 43, 45, 59] eliminate inter-phase interference but incur costly KV cache migration and load imbalance among prefill-decode instances. Bullet uniquely enables intra-GPU prefill-decode execution, maximizing utilization without overheads while meeting SLOs. Recent intra-GPU sharing proposals for LLM inference remain fundamentally constrained. Nanoflow [60] exploits kernel overlap atop chunked prefill. However, the approach’s intrinsic limitations erode gains as sequence lengths increase. MuxServe [23] and Semi-PD [30] rely on static SM allocations, requiring costly engine relaunches for reconfiguration. Drift [19] explores lock-step execution between prefill blocks and decode in gangs using predefined SM partitions. These coarse designs lack the agility required by dynamic workloads. Bullet addresses these shortcomings through adaptive fine-grained resource management, enabling optimal SM utilization across diverse workloads.

Concurrent Kernel Execution. Co-executing kernels with complementary resource demands has been proven to improve GPU utilization and performance. Spatial-temporal multiplexing techniques [13, 22, 28] typically classify kernels by computational characteristics and rely on CUDA streams [18] or MPS [39] for kernel submission. However, these approaches treat the hardware scheduler as a blackbox, providing limited control [33, 40] over actual execution. While

several systems use precise kernel timing and resource provisioning [12, 46, 56] to enforce predictable scheduling, they focus on traditional DNN workloads and fail to address LLM-specific requirements, including prefill-decode phase coordination and KV cache management. Bullet distinguishes itself from previous works by fine-grained orchestration of LLM's phases and layer-level precise resource management.

Kernel Scheduling. Effective scheduling is essential for optimizing concurrent kernel execution. Latency model-based solutions [12, 46, 56] schedule kernels within latency constraints but rely on excessive offline profiling, which is unsuitable for the dynamic workloads in LLM serving. Although estimating standalone LLM inference latency is well-established [5, 51, 54], the complex interactions between concurrent prefill and decode phases remain unstudied. Existing solutions either rely on simplified models [30] or assume contention-free execution [19]. Bullet addresses this gap by developing a profile-augmented analytical model for performance estimation and refining kernel scheduling mechanisms for LLM serving.

6 Conclusion

This paper proposes Bullet to optimize LLM serving through precise GPU resource provisioning. By carefully orchestrating spatial-temporal execution between prefill and decode phases, Bullet effectively saturates wasted GPU resources. At runtime, Bullet employs real-time task progress estimation to optimize scheduling decisions and resource allocation, enabling precise and fine-grained control over layer-level resource quotas. Experimental evaluation confirms consistent latency compliance and substantial throughput gains.

References

- [1] 2022. LlamaIndex. https://github.com/jerryjliu/llama_index.
- [2] 2025. AMD HIP Runtime API Documentation. <https://rocm.docs.amd.com/en/latest/>.
- [3] 2025. CUDA Driver API.
- [4] 2025. Nsight Systems Documentation. <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>.
- [5] Amey Agrawal, Nitin Kedia, Jayashree Mohan, et al. 2024. VIDUR: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*, Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html
- [6] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [7] Joshua Bakita and James H. Anderson. 2023. Hardware Compute Partitioning on NVIDIA GPUs. In *29th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2023, San Antonio, TX, USA, May 9-12, 2023*. IEEE, 54–66. doi:10.1109/RTAS58335.2023.00012
- [8] Joshua Bakita and James H. Anderson. 2024. Demystifying NVIDIA GPU Internals to Enable Reliable GPU Management. In *30th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2024, Hong Kong, May 13-16, 2024*. IEEE, 294–305. doi:10.1109/RTAS61025.2024.00031
- [9] Joshua Bakita and James H Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 21–1.
- [10] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: Qos awareness and increased utilization for non-preemptive accelerators in warehouse scale computers. *ACM SIGPLAN Notices* 51, 4 (2016), 681–696.
- [11] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. [n. d.]. SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines. In *THE WEB CONFERENCE 2025*.
- [12] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 199–216.
- [13] Marcus Chow, Ali Jahanshahi, and Daniel Wong. 2023. KRISP: Enabling Kernel-wise Right-sizing for Spatial Partitioned GPU Inference Servers. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*. IEEE, 624–637. doi:10.1109/HPCA56546.2023.10071121
- [14] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, et al. 2018. A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 2 (Short Papers)*, Marilyn A. Walker, Heng Ji, and Amanda Stent (Eds.). Association for Computational Linguistics, 615–621. doi:10.18653/V1/N18-2097
- [15] NVIDIA Corporation. [n. d.]. NVIDIA Deep Learning Performance. <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>.
- [16] Nvidia Corporation. 2021. Nvidia A100 tensor core GPU architecture. <https://resources.nvidia.com/en-us/genomics-ep/ampere-architecture-white-paper>.
- [17] NVIDIA Corporation. 2022. NVIDIA H100 Tensor Core GPU Architecture Overview. <https://resources.nvidia.com/en-us-tensor-core>.
- [18] NVIDIA Corporation. 2025. CUDA Runtime API Documentation. <https://docs.nvidia.com/cuda/cuda-runtime-api>.
- [19] Weihao Cui, Yukang Chen, Han Zhao, Ziyi Xu, Quan Chen, Xusheng Chen, Yangjie Zhou, Shixuan Sun, and Minyi Guo. 2025. Optimizing SLO-oriented LLM Serving with PD-Multiplexing. *arXiv preprint arXiv:2504.14489* (2025).
- [20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv:2205.14135 [cs.LG]* <https://arxiv.org/abs/2205.14135>
- [21] DeepSeek-AI, Aixin Liu, Bei Feng, et al. 2024. DeepSeek-V3 Technical Report. *CoRR* abs/2412.19437 (2024). doi:10.48550/ARXIV.2412.19437
- [22] Aditya Dhakal, Sameer G. Kulkarni, and K. K. Ramakrishnan. 2020. GSLICE: controlled spatial sharing of GPUs for a scalable inference platform. In *SoCC '20: ACM Symposium on Cloud Computing, Virtual Event, USA, October 19-21, 2020*, Rodrigo Fonseca, Christina Delimitrou, and Beng Chin Ooi (Eds.). ACM, 492–506. doi:10.1145/3419111.3421284
- [23] Jiangfei Duan, Runyu Lu, Haojie Duanmu, et al. 2024. MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=R0SoZvqXyQ>
- [24] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). doi:10.48550/ARXIV.2407.21783

- [25] Bin Gao, Zhuomin He, Puru Sharma, et al. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with Cache-dAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024*, Saurabh Bagchi and Yiyang Zhang (Eds.). USENIX Association, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [26] X Yu Geoffrey, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. 2021. Habitat: A {Runtime-Based} computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 503–521.
- [27] Guin Gilman, Samuel S. Ogden, Tian Guo, and Robert J. Walls. 2020. Demystifying the Placement Policies of the NVIDIA GPU Thread Block Scheduler for Concurrent Kernels. *SIGMETRICS Perform. Evaluation Rev.* 48, 3 (2020), 81–88. doi:10.1145/3453953.3453972
- [28] Jianfeng Gu, Yichao Zhu, Puxuan Wang, et al. 2023. FaST-GShare: Enabling Efficient Spatio-Temporal GPU Sharing in Serverless Computing for Deep Learning Inference. In *Proceedings of the 52nd International Conference on Parallel Processing, ICPP 2023, Salt Lake City, UT, USA, August 7-10, 2023*. ACM, 635–644. doi:10.1145/3605573.3605638
- [29] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>
- [30] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, et al. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. *arXiv preprint arXiv:2504.19867* (2025).
- [31] Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=VtmBAGCN7o>
- [32] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [33] Paras Jain, Xiangxi Mo, Ajay Jain, et al. 2019. Dynamic Space-Time Scheduling for GPU Inference. *CoRR* abs/1901.00041 (2019). arXiv:1901.00041 <http://arxiv.org/abs/1901.00041>
- [34] Abhinav Jangda, Saeed Maleki, Maryam Mehri Dehnavi, Madan Musuvathi, and Olli Saarikivi. 2024. A Framework for Fine-Grained Synchronization of Dependent GPU Kernels. In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2024, Edinburgh, United Kingdom, March 2-6, 2024*, Tobias Grosser, Christophe Dubach, Michel Steuwer, Jingling Xue, Guilherme Ottoni, and ernando Magno Quintão Pereira (Eds.). IEEE, 93–105. doi:10.1109/CGO57630.2024.10444873
- [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 611–626. doi:10.1145/3600006.3613165
- [36] Seonho Lee, Amar Phanishayee, and Divya Mahajan. 2025. Forecasting GPU performance for deep learning training and inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 493–508.
- [37] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, et al. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 929–945. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [38] NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>.
- [39] NVIDIA. 2025. Multi-Process Service. <https://docs.nvidia.com/deploy/mps/index.html>.
- [40] Ignacio Sanudo Olmedo, Nicola Capodiceci, Jorge Luis Martinez, et al. 2020. Dissecting the CUDA scheduling hierarchy: a Performance and Predictability Perspective. In *IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2020, Sydney, Australia, April 21-24, 2020*. IEEE, 213–225. doi:10.1109/RTAS48715.2020.000-5
- [41] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). doi:10.48550/ARXIV.2303.08774 arXiv:2303.08774
- [42] Muhammad Osama, Duane Merrill, Cris Cecka, et al. 2023. Stream-K: Work-Centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2023, Montreal, QC, Canada, 25 February 2023 - 1 March 2023*, Maryam Mehri Dehnavi, Milind Kulkarni, and Sriram Krishnamoorthy (Eds.). ACM, 429–431. doi:10.1145/3572848.3577479
- [43] Pratyush Patel, Esha Choukse, Chaojie Zhang, et al. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019
- [44] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, et al. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 1133–1150. doi:10.1145/3669940.3707256
- [45] Ruoyu Qin, Zheming Li, Weiran He, et al. 2025. Mooncake: Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies, FAST 2025, Santa Clara, CA, February 25-27, 2025*, Haryadi S. Gunawi and Vasily Tarasov (Eds.). USENIX Association, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [46] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. ACM, 1075–1092. doi:10.1145/3627703.3629578
- [47] DeepSpeed Team. 2025. DeepSpeed-MII: Enabling Low-Latency, High-Throughput Inference. <https://github.com/deepspeedai/DeepSpeed-MII>.
- [48] ShareGPT Teams. 2023. Share your wildest ChatGPT conversations with one click. <https://sharegpt.com/>
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [50] Zhenning Wang, Jun Yang, Rami G. Melhem, et al. 2016. Simultaneous Multikernel: Fine-Grained Sharing of GPUs. *IEEE Comput. Archit. Lett.* 15, 2 (2016), 113–116. doi:10.1109/LCA.2015.2477405
- [51] Bingyang Wu, Shengyu Liu, Yinmin Zhong, et al. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP 2024, Austin, TX, USA*,

- November 4-6, 2024, Emmett Witchel, Christopher J. Rossbach, Andrea C. Arpaci-Dusseau, and Kimberly Keeton (Eds.). ACM, 640–654. doi:10.1145/3694715.3695948
- [52] Ran Xu, Subrata Mitra, Jason Rahman, Peter Bai, Bowen Zhou, Greg Bronevetsky, and Saurabh Bagchi. 2018. Pythia: Improving datacenter utilization via precise contention prediction for multiple co-located workloads. In *Proceedings of the 19th International Middleware Conference*. 146–160.
 - [53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
 - [54] Zhihang Yuan, Yuzhang Shang, Yang Zhou, et al. 2024. LLM Inference Unveiled: Survey and Roofline Model Insights. *CoRR* abs/2402.16363 (2024). doi:10.48550/ARXIV.2402.16363 arXiv:2402.16363
 - [55] ZeroMQ authors. [n. d.]. ZeroMQ: An open-source universal messaging library. <https://zeromq.org/>
 - [56] Shulai Zhang, Quan Chen, Weihao Cui, Han Zhao, Chunyu Xue, Zhen Zheng, Wei Lin, and Minyi Guo. 2025. Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing. In *Proceedings of the Twentieth European Conference on Computer Systems*. 573–588.
 - [57] Yongkang Zhang, Haoxuan Yu, Chenxia Han, et al. 2025. SGDR: Software-Defined Dynamic Resource Control for Concurrent DNN Inference on NVIDIA GPUs. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2025, Las Vegas, NV, USA, March 1-5, 2025*. ACM, 267–281. doi:10.1145/3710848.3710863
 - [58] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. 2023. Efficiently Programming Large Language Models using SGLang. *CoRR* abs/2312.07104 (2023). doi:10.48550/ARXIV.2312.07104 arXiv:2312.07104
 - [59] Yinmin Zhong, Shengyu Liu, Junda Chen, et al. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
 - [60] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, et al. 2025. {NanoFlow}: Towards Optimal Large Language Model Serving Throughput. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 749–765.